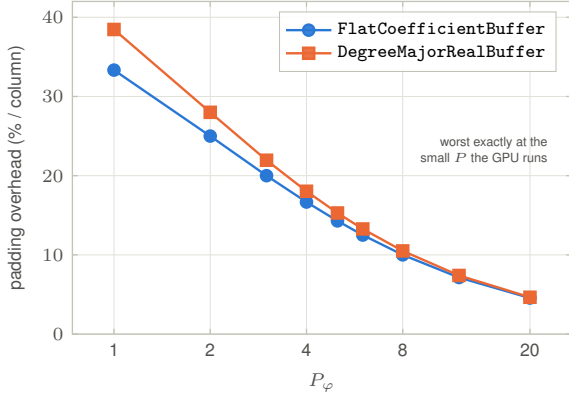
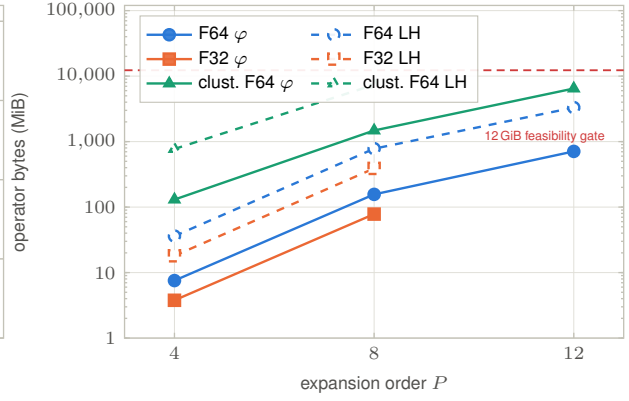


Fig. 4 — Storage and allocation: buffers, operators, caches, geometry, and warmed launches

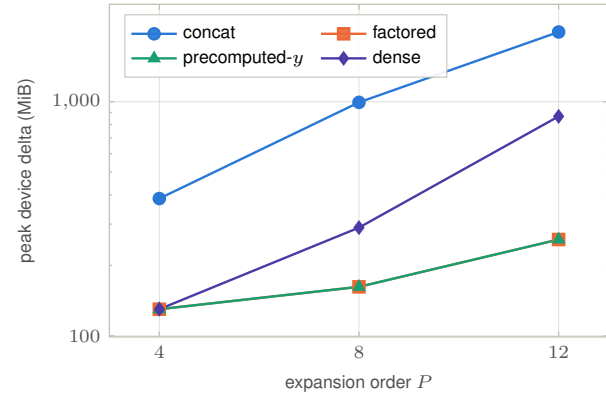
(a) padded-vs-ragged χ storage



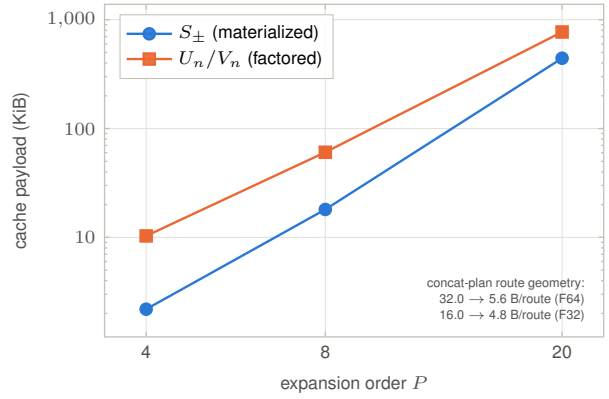
(b) DenseTranslationM2L operator payload



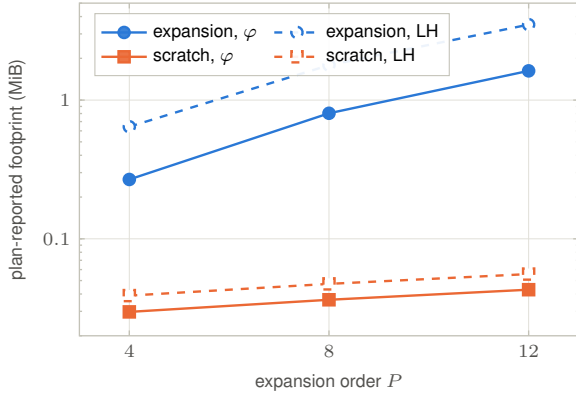
(c) H200 peak device memory



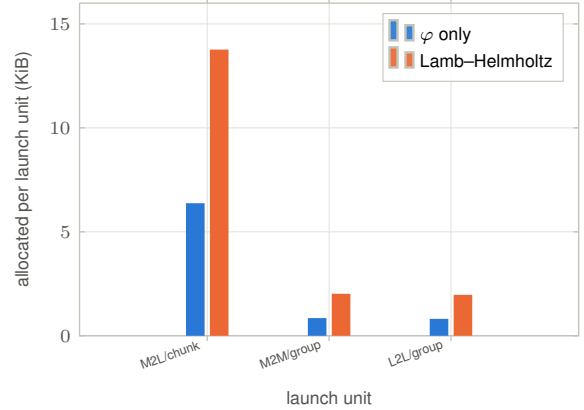
(d) 019 invariant-cache payload



(e) concat expansion and scratch buffers



(f) warmed host allocation, $P = 4$



Machines / regimes: (a) analytic, host-independent. (b,c) m13h-1-1 H200, task 024 CUDA campaign, uniform $n = 20000$ except the clustered series. (d,f) local macOS measurement, task 019. Panel (e) uses the task-024 H200 concat plan at uniform $n = 20000$, Float64.

Data: smallp_fallback_layout/m12-1-7/layout_storage.csv, operator_ab_benchmark/summary/memory_feasibility.csv, operator_performance_tuning/local_macos_allocations_storage.csv (tables fig04a*...fig04g*). **Reads as:** padding χ to P_φ costs 17–33% (flat) / 18–39% (degree-major) extra storage per column at $P \leq 4$ — worst precisely where the GPU operates — which is why the ragged layout was kept. The dense strategy's operator payload grows by roughly $20\times$ per doubling of P and crosses the 12 GiB gate for Lamb-Helmholtz at $P = 12$ under clustering (30.1 GiB) — the 21 infeasible cases of task 024. Panel (c) is the counter-intuitive part: for φ -only $n = 20000$ the *highest* measured device peak is the whole-slab concat path (405 $\rightarrow$ 2082 MiB), not dense (136 $\rightarrow$ 908 MiB), because concat's peak is dominated by slab scratch rather than by operators, while factored and precomputed- y stay flat at 136–271 MiB. Dense becomes the memory-limited strategy only once P , Lamb-Helmholtz or clustering inflate its operator payload past the gate, as in panel (b). Panel (e) separates the concat plan's coefficient storage from its reported operator scratch: expansion storage grows from 0.27 to 1.62 MiB for φ and from 0.64 to 3.50 MiB for Lamb-Helmholtz over $P = 4 \rightarrow 12$, while this plan's reported scratch remains 0.03–0.06 MiB. Panel (f) records the remaining warmed host dynamic overhead after tuning: 6.4 KiB per M2L chunk and 0.8–0.9 KiB per M2M/L2L group for φ , rising to 13.8 and about 2.0 KiB with χ . **Caveats:** panel (c) uses `peak_bytes`, which on CUDA is a per-strategy device-memory delta and therefore comparable; the CPU `peak_bytes` column is a process-wide `Sys.maxrss()` shared by all four strategies and is deliberately not plotted. Likewise `persistent_bytes/scratch_bytes` are plan-reported for dense and precomputed- y but fall back to `Base.summarysize` for concat and factored, so panel (e) is deliberately a within-concat order/channel view, not a cross-strategy scratch comparison. Panel (f)'s unit is stage-specific (chunk for M2L, group for M2M/L2L), as recorded by the source campaign; it compares warmed launch overhead, not total stage allocation. Only dense reports a non-zero `operator_bytes`: the other three build no materialized per-class operator. In panel (c) the factored and precomputed- y curves coincide exactly at all three orders (136/170/271 MiB) — the device peak is quantised by the CUDA allocation pool at this scale, so the orange squares sit underneath the green triangles. Dense Float32 at $P = 12$ is a hard-coded skip in the 024 harness, not an observed failure, so that series ends at $P = 8$.